Hanzo IAM: Identity, Access Management, and API Gateway for Multi-Tenant Cloud Applications

Zach Kelling

Hanzo AI

February 2019

Abstract

We describe the Hanzo identity, access management (IAM), and API gateway architecture, developed between 2016 and 2019. The system comprises an API gateway built on KrakenD (`hanzo/gateway`, forked December 2017), an OAuth 2.0/OIDC identity provider with multi-tenant organization support, an OpenAPI-first contract layer (2016), and a role-based access control (RBAC) system with per-organization data isolation. The IAM layer authenticates and authorizes every request entering the Hanzo platform, extracts the organization from the JWT `owner` claim, and scopes all downstream data queries to that organization. This architecture now serves as the unified identity layer for all Hanzo, Lux, and Zoo deployments, as well as white-label deployments under custom domains.

1 Introduction

Multi-tenant cloud platforms require a centralized identity layer that provides authentication, authorization, and tenant isolation across all services. Without such a layer, each service must independently implement credential validation, permission checking, and data scoping—leading to inconsistent security posture and duplicated effort.

The Hanzo IAM system was developed to serve as this centralized layer. Its evolution proceeded through three phases:

1. **OpenAPI-first design** (2016): API contracts defined before implementation, with schema-derived validation and client generation.
2. **API Gateway** (December 2017): A KrakenD-based gateway providing JWT validation, rate limiting, and request routing at the edge.
3. **Identity provider** (February 2019): An OAuth 2.0/OIDC-compliant identity provider with multi-tenant organizations and RBAC.

2 API Gateway

2.1 Architecture

The Hanzo API Gateway (`hanzo/gateway`) is built on KrakenD [3], forked in December 2017. The gateway operates as a stateless reverse proxy at the edge of the platform, handling JWT validation, rate limiting, request aggregation, and protocol translation.

2.2 Request Pipeline

Every request traverses the following pipeline:

1. **TLS termination**: Handled by the upstream ingress layer.
2. **Identity header stripping**: The gateway removes all client-supplied identity headers (`X-Org-Id`, `X-User-Id`, `X-User-Email`) before processing. Only the JWT-validated path may set these headers.
3. **JWT validation**: The gateway verifies the JWT signature against the IAM provider's JWKS endpoint, checks expiration, audience, and issuer claims.
4. **Claim extraction**: The `sub` (user ID), `email`, and `owner` (organization) claims are extracted and injected as internal headers.
5. **Rate limiting**: Per-organization and per-user rate limits are enforced.
6. **Routing**: The request is forwarded to the appropriate backend service.

2.3 Configuration

The gateway is configured declaratively via a JSON specification:

Listing 1: Gateway endpoint configuration.

```
{
  "endpoint": "/v1/products",
  "method": "GET",
  "backend": [
    {
      "host": ["http://commerce:8080"],
      "url_pattern": "/v1/products"
    }
  ],
  "extra_config": {
    "auth/validator": {
      "alg": "RS256",
      "jwk_url": "https://hanzo.id/.well-known/jwks.json",
      "audience": ["hanzo-platform"],
      "issuer": "https://hanzo.id"
    }
  }
}
```

3 Identity Provider

3.1 OAuth 2.0 and OIDC

The Hanzo identity provider implements the OAuth 2.0 Authorization Framework [1] and OpenID Connect Core [2]. Supported grant types:

- **Authorization Code**: Standard web application flow with PKCE.
- **Client Credentials**: Service-to-service authentication.
- **Device Authorization**: CLI and device-based flows.
- **Refresh Token**: Long-lived session maintenance.

3.2 OIDC Endpoints

All endpoints follow RFC 6749 and OIDC discovery:

Table 1: OIDC endpoint paths.

Function	Path
Discovery	`/.well-known/openid-configuration`
Authorize	`/oauth/authorize`
Token	`/oauth/token`
UserInfo	`/oauth/userinfo`
Introspect	`/oauth/introspect`
Revoke	`/oauth/revoke`
Device	`/oauth/device`
Logout	`/oauth/logout`

3.3 JWT Structure

Access tokens are signed JWTs containing identity and authorization claims:

Listing 2: JWT payload structure.

```
{
  "sub": "user_abc123",
  "email": "user@example.com",
  "owner": "org-acme",
  "aud": ["hanzo-platform"],
  "iss": "https://hanzo.id",
  "iat": 1550000000,
  "exp": 1550003600,
  "roles": ["admin"],
  "permissions": ["products:read", "products:write"]
}
```

The `owner` claim identifies the organization. All downstream services scope data queries to this organization.

4 Multi-Tenant Organizations

4.1 Organization Model

Each organization is an isolated tenant with its own users, roles, applications, and data:

$$\text{Org} = (\text{id}, \text{name}, \text{domain}, \text{members}, \text{roles}, \text{applications}) \tag{1}$$

Users may belong to multiple organizations. A user's effective permissions are the union of their role assignments within the active organization.

4.2 Data Isolation

The `owner` claim from the JWT is injected into every database query as a mandatory filter:

Listing 3: Org-scoped query pattern.

```go
func ListProducts(ctx context.Context) ([]Product, error) {
    orgID := ctx.Value("orgID").(string)
    return db.Query(
        "SELECT * FROM products WHERE org_id = $1",
        orgID,
    )
}
```

Services never expose data from one organization to another. The gateway is the sole source of the organization identifier; backend services do not trust client-supplied values.

5 Role-Based Access Control

5.1 Permission Model

Permissions are defined as triples of (resource, action, scope):

$$P = \{(r, a, s) \mid r \in \mathcal{R}, a \in \mathcal{A}, s \in \{\text{own}, \text{all}\}\} \tag{2}$$

where $\mathcal{R}$ is the set of resource types and $\mathcal{A} = \{\text{read}, \text{write}, \text{delete}\}$.

5.2 Role Hierarchy

Standard roles are provided:

Table 2: Default role hierarchy.

Role	Capabilities
`owner`	Full control, manage members and billing
`admin`	All resource operations, manage members
`editor`	Read and write all resources
`viewer`	Read-only access to all resources

Custom roles can be defined per organization with arbitrary permission sets.

5.3 Authorization Check

$$\text{authorized}(u, r, a) = \exists\, \rho \in \text{roles}(u, o) : (r, a, *) \in \text{permissions}(\rho) \tag{3}$$

where u is the user, r the resource type, a the action, and o the active organization.

6 White-Label Deployments

The IAM system supports white-label deployments where the identity provider serves multiple brands from a single instance. Brand detection is hostname-based: the login page, logo, colors, and email templates are determined by the domain the user accesses.

Table 3: White-label brand mapping.

Domain	Brand
`hanzo.id`	Hanzo
`lux.id`	Lux
`zoo.id`	Zoo

The underlying identity data is shared: a user authenticated at `lux.id` with an organization membership on the Hanzo platform can access Hanzo services with their Lux-issued JWT, provided the audience claim matches.

7 Service-to-Service Authentication

Internal services authenticate using the Client Credentials grant. Each service is registered as an application with a `clientId` and `clientSecret`:

Listing 4: Service authentication.

```
POST /oauth/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id=svc-commerce
&client_secret=SECRET
&audience=hanzo-platform
```

The resulting access token contains a `sub` identifying the service and scoped permissions appropriate for inter-service communication.

8 Security Considerations

- **Password storage**: All passwords are hashed with bcrypt (cost factor 12). Plaintext passwords are never stored.
- **Token signing**: RS256 with 2048-bit RSA keys, rotated quarterly. JWKS endpoint serves the current and previous public keys to allow graceful rotation.
- **Brute force protection**: Account lockout after 5 failed attempts with exponential back-off.
- **Secret management**: Client secrets and signing keys are stored in the Hanzo KMS, not in configuration files or environment variables.

9 Conclusion

The Hanzo IAM system provides a centralized identity, access management, and API gateway layer for the entire Hanzo platform. By extracting the organization from JWT claims and enforcing data isolation at the query level, the system ensures strict multi-tenant separation. The gateway strips client-supplied identity headers and relies solely on JWT-validated claims, preventing privilege escalation. OAuth 2.0/OIDC compliance enables integration with standard tooling, while white-label support allows the same identity infrastructure to serve multiple brands. This architecture now provides authentication and authorization for all Hanzo, Lux, and Zoo services in production.

References

[1] D. Hardt. The OAuth 2.0 Authorization Framework. RFC 6749, IETF, 2012.

[2] N. Sakimura et al. OpenID Connect Core 1.0. OpenID Foundation, 2014.

[3] KrakenD. KrakenD: Ultra-High Performance API Gateway. `https://krakend.io`, 2017.

[4] M. Jones, J. Bradley, and N. Sakimura. JSON Web Token (JWT). RFC 7519, IETF, 2015.

[5] M. Jones. JSON Web Key (JWK). RFC 7517, IETF, 2015.